

Bài Mô Phỏng Đầu Tiên Cùng Webots

Hoạt động

1. Làm quen Giao diện webots + **Thao tác với chuột**

2. Tạo thế giới mới cho riêng mình

3. Thêm **robot e-puck** hay bất kỳ robot nào em thích

4. Lập trình **controller** cơ bản

Đối tượng hướng tới: Học sinh có đam mê sau khóa học ROBO001/ sinh viên làm quen mô phỏng robot (nắm cơ bản về python/ c++).

Lưu ý: Yêu cầu các bạn tham khảo hướng dẫn cơ bản về cách sử dụng webots

I. Tạo thế giới mới cho riêng mình

A. Khái niệm về world (thế giới mô phỏng)

World (Thế giới) là một tệp tin chứa toàn bộ thông tin như: vị trí các vật thể, hình dáng của chúng, cách chúng tương tác với nhau, màu sắc của bầu trời, cũng như các định nghĩa về trọng lực, ma sát, khối lượng của vật thể... Nó xác định **trạng thái ban đầu** của một phiên mô phỏng.

Các cấu trúc cơ bản:

- + **Nodes (Nút):** Các vật thể khác nhau trong thế giới được gọi là các Node.
- + **Scene Tree (Cây phối cảnh):** Các Node được sắp xếp theo cấu trúc phân cấp trong một Scene Tree. Do đó, một Node có thể chứa các Node con bên trong nó.
- + **Định dạng tệp:** Thế giới được lưu trữ dưới dạng tệp có phần mở rộng là **.wbt**. Định dạng này được kế thừa từ ngôn ngữ **VRML97** và con người có thể đọc/hiểu được nội dung bên trong (dạng văn bản). File **.wbt** hoàn toàn có thể mở và chỉnh sửa bằng code editor như VSCode, Notepad++, v.v.

- + **Lưu trữ:** Các tệp world phải được lưu trữ trực tiếp trong một thư mục có tên là **worlds**.

File .wbt hoàn toàn có thể mở và chỉnh sửa bằng code editor như VSCode, Notepad++, v.v.

```

1 #VRML_SIM R2023b utf8
2
3 EXTERNPROTO "https://raw.githubusercontent.com/cyberbotics/webots/R2023b/projects/objects/backgrounds/protos/TexturedBackground.proto"
4 EXTERNPROTO "https://raw.githubusercontent.com/cyberbotics/webots/R2023b/projects/objects/backgrounds/protos/TexturedBackgroundLight.proto"
5 EXTERNPROTO "https://raw.githubusercontent.com/cyberbotics/webots/R2023b/projects/robots/gctronic/e-puck/protos/E-puck.proto"
6 EXTERNPROTO "https://raw.githubusercontent.com/cyberbotics/webots/R2023b/projects/objects/floors/protos/RectangleArena.proto"
7
8 WorldInfo {
9 }
10 Viewpoint {
11   orientation 0.32153774255619283 0.048545132077401125 -0.9456515480151528 3.0486494200862606
12   position 1.3379505993171432 0.05904384182527796 1.1073111182309785
13 }
14 TexturedBackground {
15 }
16 TexturedBackgroundLight {
17 }
18 E-puck {
19   controller "custom_robot_window_simple"
20   window "custom_robot_window_simple"
21 }
22 RectangleArena {
23 }
24

```

VS Code

```

custom_robot_window_simple.wbt - Notepad
File Edit Format View Help
#VRML_SIM R2023b utf8

EXTERNPROTO "https://raw.githubusercontent.com/cyberbotics/webots/R2023b/projects/objects/backgrounds/protos/TexturedBackground.proto"
EXTERNPROTO "https://raw.githubusercontent.com/cyberbotics/webots/R2023b/projects/objects/backgrounds/protos/TexturedBackgroundLight.proto"
EXTERNPROTO "https://raw.githubusercontent.com/cyberbotics/webots/R2023b/projects/robots/gctronic/e-puck/protos/E-puck.proto"
EXTERNPROTO "https://raw.githubusercontent.com/cyberbotics/webots/R2023b/projects/objects/floors/protos/RectangleArena.proto"

WorldInfo {
}
Viewpoint {
  orientation 0.32153774255619283 0.048545132077401125 -0.9456515480151528 3.0486494200862606
  position 1.3379505993171432 0.05904384182527796 1.1073111182309785
}
TexturedBackground {
}
TexturedBackgroundLight {
}
E-puck {
  controller "custom_robot_window_simple"
  window "custom_robot_window_simple"
}
RectangleArena {
}

```

Notepad

Bạn có biết: Mọi thứ ta thấy trên màn hình 3D: robot vật thể, vị trí, kích thước, đều được mô tả bằng văn bản.

Bạn có thể:

1. Copy / paste nhanh cả một cụm vật thể,
2. nhân bản robot hoặc chương ngại vật chỉ bằng vài dòng code thôi ghê chưa

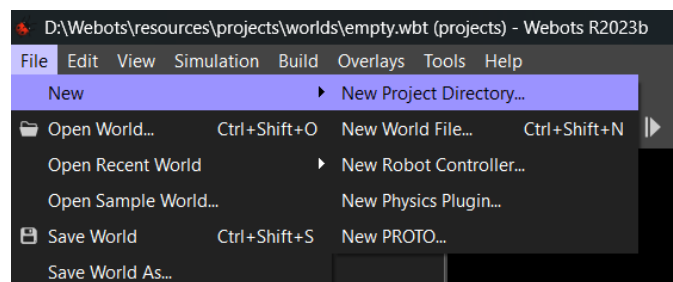
B. Thực hành tạo thế giới

Bước 1: Mở webots trên màn hình (hoặc bằng terminal)

*Lưu ý: Đối với các bạn chưa tải webots, chúng ta hãy thực hiện download phần mềm về theo hướng dẫn này nhé:

 Webot Installation & Simple Usage Guide (2025).pptx (Slide số 5)

Bước 2: Thực hiện tạo thế giới mới lần lượt theo hướng dẫn sau: Nháy chọn nút File ở góc trái màn hình của webots -> Nhấn vào nút New -> Chọn New Project Directory



Bước 3: Nhấn Next để Wizard hỗ trợ chọn đường dẫn lưu trữ dự án. Ở bước này, ta cần tạo 1 folder mới ở ổ D (hoặc ổ E) để chứa dự án mới thay vì chọn đường dẫn như webots đề xuất

Bước 4: Đặt tên cho project mới thay vì để my project. Sau cùng là nhấn Finish.

*Lưu ý: Hãy tick hết vào các ô trong đó có cả ô “Add a rectangle arena” do ô này không được tick sẵn trước đó.

Note: Không lưu project trong thư mục mặc định của Webots

Khi mở Sample World, luôn sử dụng chức năng Save World As... để lưu thành một bản sao mới trước khi chỉnh sửa.

Việc này tránh làm hỏng các file mẫu mặc định của Webots.

C. Kiểm tra thế giới mới

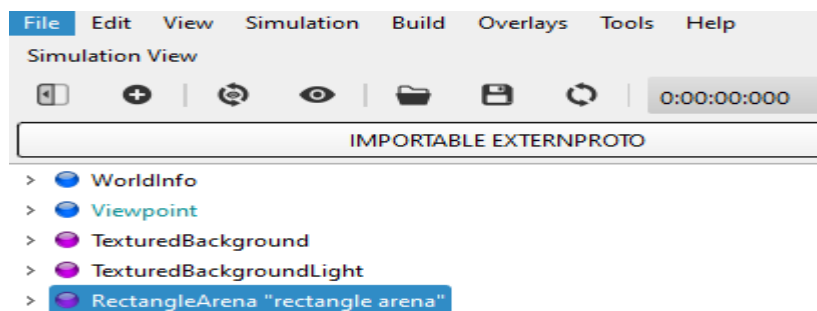
Chúc mừng bạn đã tạo ra thế giới mới của riêng mình!!! Lúc này, cửa sổ xem 3D (3D view) sẽ hiển thị một đấu trường hình vuông với sàn nhà kẻ ô ca-rô. Bạn có thể thay đổi góc nhìn trong cửa sổ này bằng chuột: **chuột trái, chuột phải và con lăn**. Tiếp theo, hãy cùng mình điểm qua các đặc trưng của thế giới mới nhé:

1/ Các nút (**Nodes**) trong Webots được lưu trữ trong tệp thế giới và sắp xếp theo cấu trúc cây, gọi là **Scene Tree** (Cây phối cảnh). Cây phối cảnh này có thể được quan sát qua hai cửa sổ phụ:

- + **3D View (ở giữa)**: Là hình ảnh đại diện 3D của cây phối cảnh.
- + **Scene Tree View (bên trái)**: Là một biểu đồ phân cấp. Đây là nơi bạn có thể chỉnh sửa các Node và các trường dữ liệu (**Fields**).

2/ Với một thế giới mới tạo, danh sách các Nodes mà bạn hiện có sẽ là:

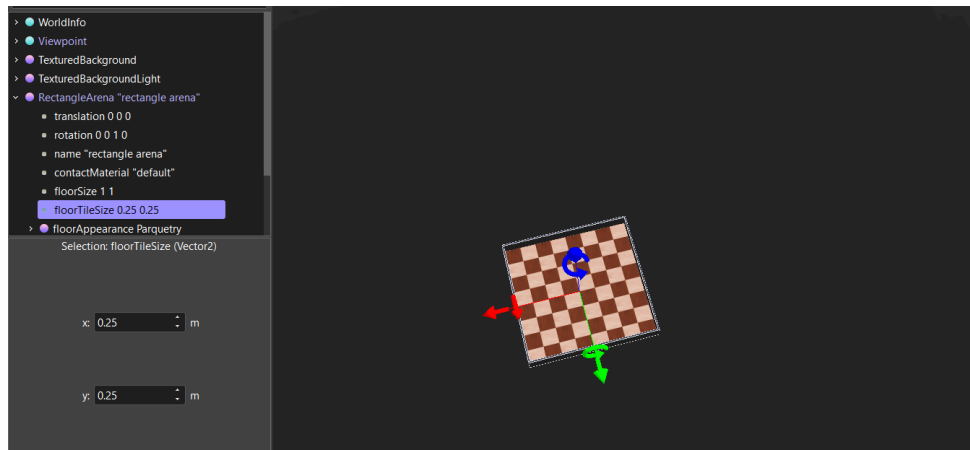
- + WorldInfo: Chứa các thông số toàn cục của hệ thống mô phỏng.
- + Viewpoint: Xác định các thông số camera cho màn hình chính.
- + TexturedBackground: Xác định phong nền của cảnh (bạn sẽ thấy núi ở phía xa nếu xoay góc nhìn một chút).
- + TexturedBackgroundLight: Xác định ánh sáng đi kèm với phong nền phía trên.
- + RectangleArena: Xác định vật thể duy nhất mà bạn thấy trong cảnh cho đến lúc này.



3/ Mỗi Node đều có các thuộc tính có thể tùy chỉnh được gọi là **Fields**. Hãy cùng thực hành một chút bằng cách thử thay đổi các trường này để biến đổi dấu trường hình chữ nhật nhé:

Bước 1: Nháy đúp chuột vào Node **RectangleArena** trong cửa sổ Scene Tree. Thao tác này sẽ mở rộng Node đó và hiển thị các trường dữ liệu (**fields**) bên trong.

Bước 2: Chọn trường **floorTileSize** và đặt giá trị của nó thành **0.25 0.25** thay vì **0.5 0.5** như ban đầu. Bạn sẽ thấy hiệu ứng thay đổi ngay lập tức ở cửa sổ 3D View (các ô gạch dưới sàn sẽ nhỏ lại).



Hình minh họa

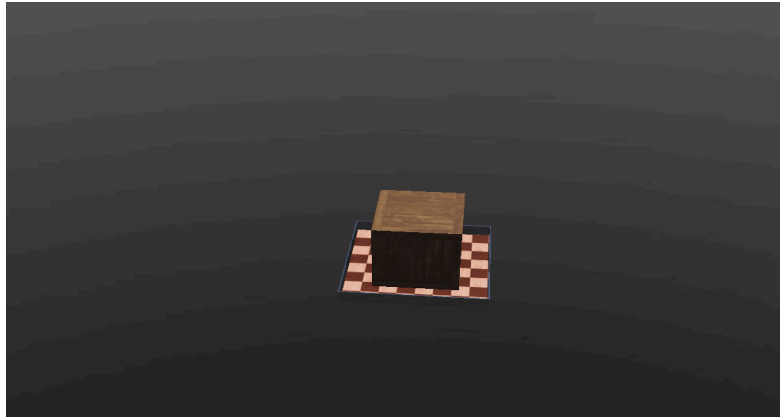
Bước 3: Chọn trường **wallHeight** và thay đổi giá trị thành **0.05** thay vì **0.1**. Lúc này, tường bao quanh đấu trường sẽ thấp xuống.

D. Thực Hành Thả Vật Thể Vào Không Gian 3D

Trong giao diện của Scene Tree, các trường dữ liệu được biểu diễn dưới nhiều màu khác nhau (phụ thuộc vào màu nền của chúng) nếu như chúng khác với các giá trị mặc định ban đầu. Nối tiếp với bài thực hành vừa rồi, bây giờ chúng ta hãy cùng thực hành đưa vật thể (khối gỗ) vào trong thế giới nhé!

Bước 1: Thêm vật thể

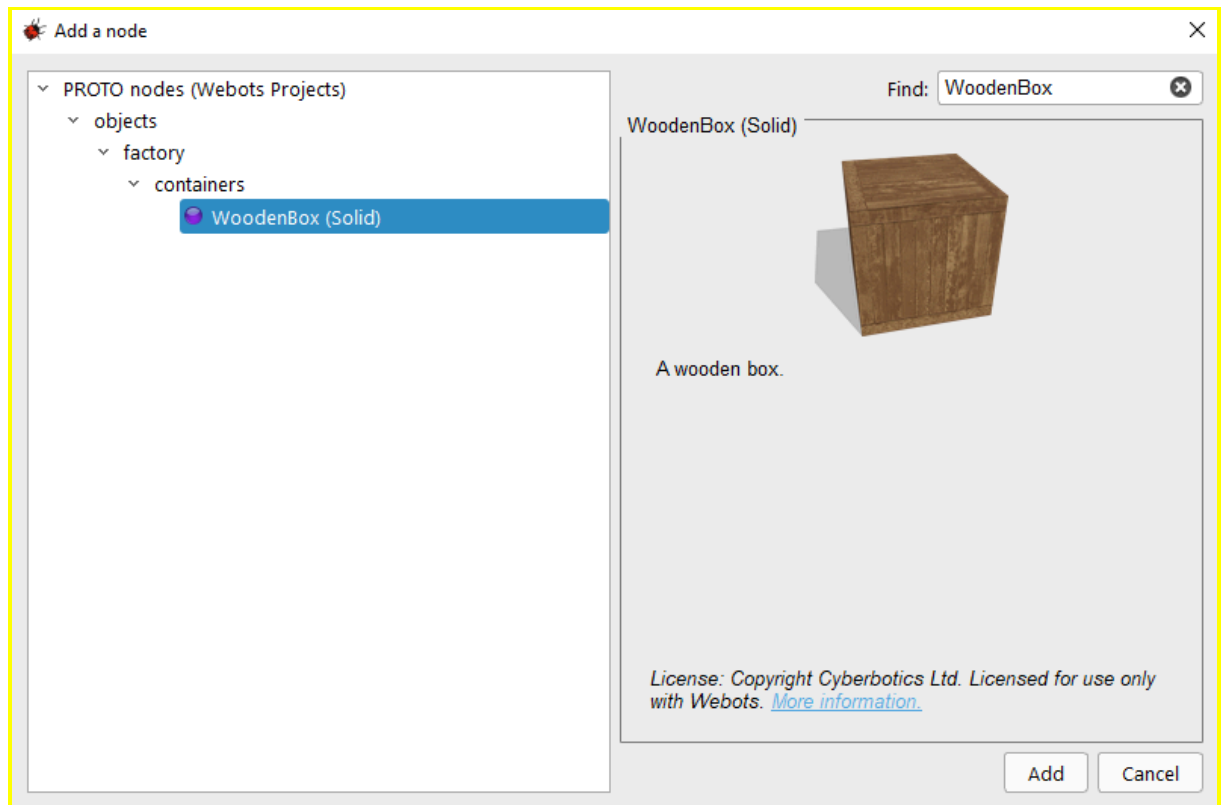
- + Nhấp đúp chuột vào **RectangleArena** trong Scene Tree để đóng nó lại và chọn nó.
- + Nhấn vào nút **Add** (biểu tượng dấu cộng) ở phía trên cùng của cửa sổ Scene Tree.
- + Trong hộp thoại vừa mở, chọn theo đường dẫn: **PROTO nodes (Webots Projects) / objects / factory / containers / WoodenBox(Solid)**. Một chiếc thùng gỗ lớn sẽ xuất hiện ở giữa đấu trường.



Hình minh họa khi thả woodenbox thành công

Tips:

Các em có thể tìm gỗ WoodenBox cũng được nhé



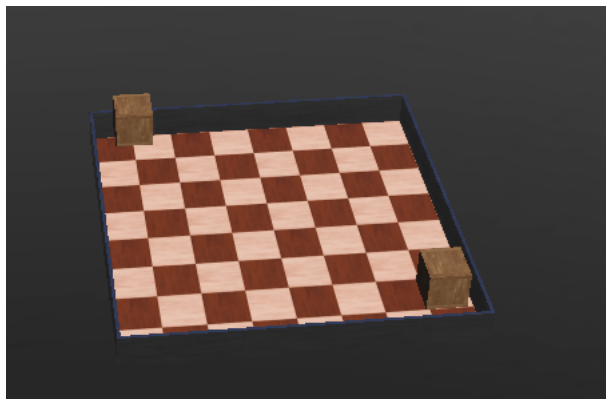
Bước 2: Chỉnh kích thước và vị trí

- + Nhấp đúp vào **WoodenBox** trong Scene Tree để mở các trường dữ liệu của nó.
- + Thay đổi **size** (kích thước) thành **0.1 0.1 0.1** (thay vì 0.6 0.6 0.6).
- + Thay đổi **translation** (vị trí) thành **0 0 0.05** (thay vì 0 0 0.3). Ngoài

ra, bạn có thể dùng mũi tên màu xanh dương xuất hiện trong cửa sổ 3D để điều chỉnh giá trị translation.z.

Bước 3: Di chuyển và Sao chép

- + Nhấn giữ **Shift + chuột trái** và kéo chiếc thùng trong cửa sổ 3D để đưa nó vào một góc của đấu trường.
- + Chọn chiếc thùng và nhấn **Ctrl+C**, **Ctrl+V** (Windows/Linux) hoặc **⌘+C**, **⌘+V** (macOS) để sao chép và dán. Sau đó, nhấn giữ Shift và kéo chiếc thùng mới sang vị trí khác. Hãy tạo chiếc thùng thứ hai theo cách tương tự.



Hình minh họa

Bước 4: Sắp xếp và Xoay

- + Di chuyển các thùng sao cho không có chiếc nào nằm ở giữa đấu trường.
- + Bạn có thể dùng các mũi tên xoay màu xanh dương để xoay thùng quanh trục thẳng đứng. Cách khác là nhấn giữ **Shift + chuột phải** và kéo chuột, hoặc thay đổi thông số góc (angle) trong trường **rotation** của các nút [WoodenBox](#) trong Scene Tree.

Bước 5: Lưu lại

- + Khi đã hài lòng với kết quả, hãy nhấn nút **Save** để lưu lại thể giới mô phỏng của bạn.

II. Thêm Robot E-puck vào dự án của bạn

A. Giới thiệu về E-puck Robot

E-puck là một robot nhỏ sở hữu hệ thống bánh xe vi sai (differential wheels), 10 đèn LED và nhiều loại cảm biến khác nhau,

bao gồm 8 cảm biến khoảng cách (**DistanceSensors**) và một Camera. Trong bài hướng dẫn này, chúng ta sẽ chỉ tập trung vào việc sử dụng bánh xe của nó. Các tính năng khác sẽ được tìm hiểu trong các bài học kế tiếp.

B.Thêm E-puck Robot vào thế giới

1/ Phần Chuẩn Bị:

Bây giờ, chúng ta sẽ thêm mô hình e-puck vào thế giới mô phỏng. Hãy đảm bảo thực hiện theo các quy tắc quan trọng sau:

+ **Chuẩn bị trạng thái:** Đảm bảo rằng phiên mô phỏng đang được tạm dừng (**pause**) và thời gian ảo đã trôi qua là **0**. Nếu không phải vậy, hãy nhấn nút **Reset** để đưa mọi thứ về trạng thái bắt đầu.

+ **Quy trình bắt buộc khi chỉnh sửa:** Khi bạn muốn sửa đổi và lưu lại một "thế giới" trong Webots, điều cực kỳ quan trọng là phải **tạm dừng và tải lại (reset)** về trạng thái ban đầu (đồng hồ thời gian trên thanh công cụ phải hiển thị **0:00:00:000**).

* Lưu ý: Nếu không làm vậy, với mỗi lần chạy mô phỏng, các vật thể sẽ bị tích tụ sai số. Bên cạnh đó, các thứ tự thực hiện các hành động cũng phải tuân theo thứ tự: Pause -> Reset -> Modify -> Save

Chúng ta sẽ không cần phải tạo robot e-puck từ con số không, mà chỉ cần nhập một Node gọi là **E-puck**. Nút này thực chất là một node **PROTO** (tương tự như **WoodenBox**). Việc tạo mẫu (Prototyping) này cho phép bạn tạo ra các vật thể tùy chỉnh và tái sử dụng chúng một cách dễ dàng.

2/ Phần Thực Hành: Thêm E-puck Robot vào thế giới

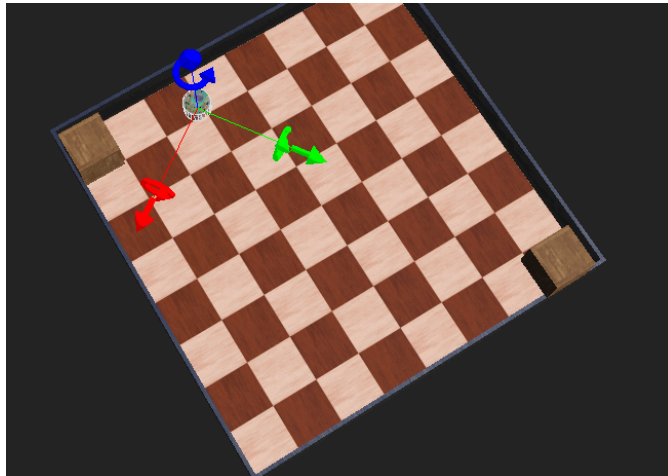
Bước 1: Tìm và thêm robot

- Chọn nút **WoodenBox** cuối cùng trong danh sách Scene Tree.
- Nhấn vào nút **Add** (biểu tượng dấu cộng) ở phía trên cùng.
- Trong hộp thoại hiện ra, hãy chọn theo đường dẫn: **PROTO nodes (Webots Projects) / robots / gctronic / e-puck / E-puck (Robot)**. Một chú robot e-puck sẽ xuất hiện ngay giữa đấu trường.

Bước 2: Điều chỉnh vị trí: Thực hiện tương tự với **WoodenBox** để đưa robot đến vị trí mong muốn.

Bước 3: Chạy mô phỏng: Lưu lại phiên mô phỏng (Save) và

nhấn nút Run real-time (biểu tượng nút Play) để bắt đầu quá trình mô phỏng



Hình minh họa

Mô tả hoạt động: Robot sẽ bắt đầu di chuyển, nhấp nháy đèn LED và tránh các chướng ngại vật. Sở dĩ có điều này là vì nó đã được thiết lập một **bộ điều khiển mặc định (default controller)** với các hành vi nêu trên. Ngoài ra, lúc này ở góc trái của phía trên màn hình 3D bạn có thể sẽ thấy một cửa sổ nhỏ màu đen, đó là hình ảnh thu được từ Camera của Robot E-puck và hình ảnh này vẫn sẽ là màu đen trừ khi camera của robot được kích hoạt.

Tips: Bạn bất kỳ robot nào bạn thích nhé. Chọn trong mục Nodes--robot



Thao tác với cửa sổ Camera:

- Di chuyển: Bạn có thể kéo cửa sổ này đến bất cứ đâu trong khung nhìn 3D.
- Thay đổi kích thước: Kéo góc dưới bên phải để phóng to hoặc thu nhỏ.
- Đóng cửa sổ: Nhấp vào dấu "x" ở góc trên bên phải.
- **Hiển thị lại:** Nếu đã lỡ đóng, bạn có thể làm nó hiện lại bằng cách vào menu **Overlays** chọn mục checkbox trong menu con **Hide All Camera Devices**.

*Lưu ý: Do chưa cần Camera nên ta sẽ tạm ẩn nó đi nhé

C.Thực Hành Các Tác Dụng Vật Lý

Trong khi phần Simulation vẫn đang chạy, chúng ta hãy cùng thử chút với các tính năng vật lý nhé! Chúng ta sẽ tác động vật lý bằng chuột:

Trên Windows: Nhấn giữ **Alt + chuột trái + kéo chuột**.

Trên Mac: Nhấn giữ **Option (⌘) + chuột trái + kéo chuột**.

Trên Linux: Nhấn giữ **Ctrl + Alt + chuột trái + kéo chuột**.

Lưu ý rằng ta sẽ không thể tác dụng vật lý lên các WoodenBox do các cài đặt mặc định của chúng là không có khối lượng và được xem như dán chặt vào không gian. Để kích hoạt, chúng ta sẽ phải vào trường mass (khối lượng) và đặt cho nó một

giá trị, ví dụ như 0.2kg. Sau khi thực hiện xong, chúng ta có thể đẩy hoặc kéo vật thể như robot



Hình minh họa

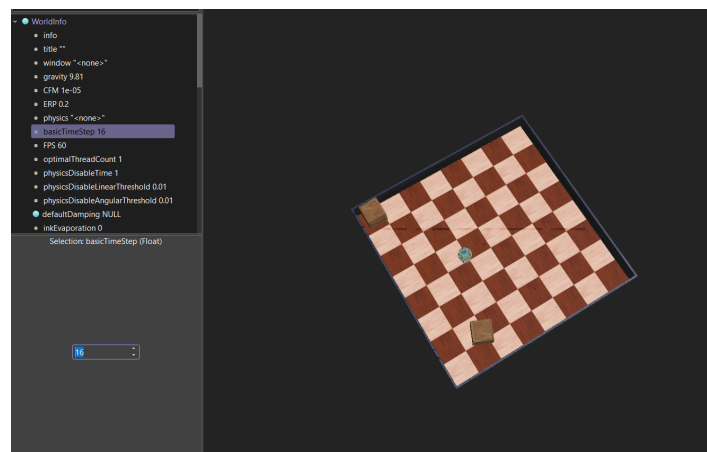
Bây giờ chúng ta sẽ chỉnh sửa thể giới để giảm bước thời gian (step) của mô phỏng vật lý: việc này sẽ giúp tăng **độ chính xác** và **độ ổn định** của mô phỏng (nhưng sẽ làm giảm tốc độ mô phỏng tối đa).

Bước 1: Tạm dừng mô phỏng và nhấn nút **Revert** (để đưa mọi thứ về trạng thái ban đầu).

Bước 2: Trong cửa sổ Scene Tree, hãy mở rộng nút **WorldInfo** (nút đầu tiên trong danh sách).

Bước 3: Tìm trường **basicTimeStep** và đặt giá trị của nó thành **16** (đơn vị là miligiây).

Bước 4: Lưu lại (Save) phiên mô phỏng.



Hình minh họa

III. Tạo Một Bộ Điều Khiển (Controller)

A. Giới thiệu bộ điều khiển

Bây giờ chúng ta sẽ lập trình một bộ điều khiển (controller) đơn giản để điều khiển robot di chuyển về phía trước.

Controller là một chương trình xác định hành vi của robot. Trong Webots, bộ điều khiển có thể được viết bằng nhiều ngôn ngữ lập trình khác nhau như: **C, C++, Java, Python, MATLAB**, v.v.

Dưới đây là một số lưu ý quan trọng về ngôn ngữ lập trình:

- **Ngôn ngữ cần biên dịch:** Đối với **C, C++ và Java**, bạn cần phải biên dịch (compile) mã nguồn thành file thực thi trước khi có thể chạy chúng trên robot.
- **Ngôn ngữ thông dịch: Python và MATLAB** là các ngôn ngữ thông dịch, vì vậy chúng có thể chạy trực tiếp mà không cần bước biên dịch.

Trong bài hướng dẫn này, chúng ta sẽ sử dụng **C** làm ngôn ngữ tham chiếu. Tuy nhiên, tất cả các đoạn mã mẫu đều có sẵn phiên bản dành cho C++, Java, Python và MATLAB (nằm trong webots documentation). Nếu bạn muốn thiết lập bộ điều khiển bằng một ngôn ngữ khác, hãy tham khảo chương về ngôn ngữ lập trình trong tài liệu của Webots.

B. Những lệnh cơ bản để tạo bộ điều khiển

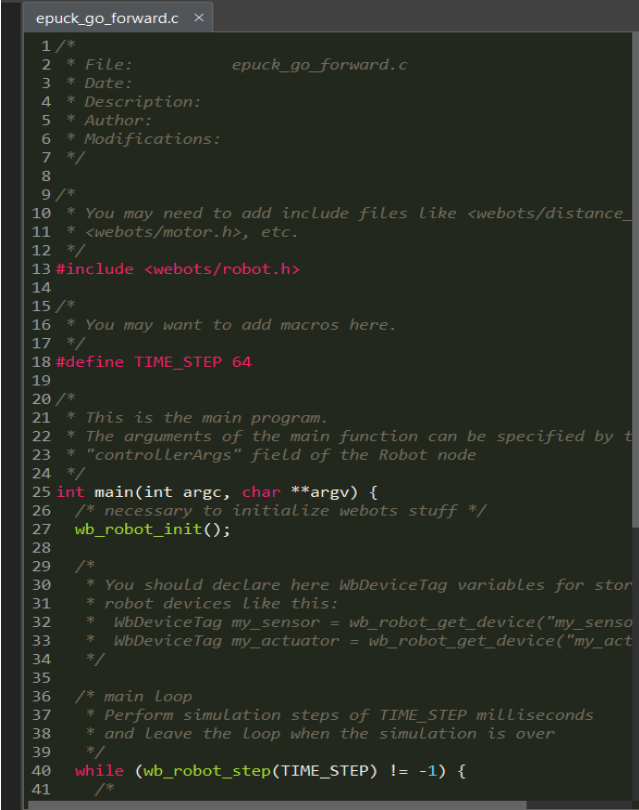
Trường **controller** của một nút **Robot** xác định bộ điều khiển nào hiện đang được gắn với robot đó. Lưu ý rằng:

- Một bộ điều khiển có thể được dùng cho nhiều robot cùng lúc.
- Tuy nhiên, một robot chỉ có thể sử dụng **duy nhất một** bộ điều khiển tại một thời điểm.
- Mỗi bộ điều khiển được thực thi trong một tiến trình con riêng biệt (thường do Webots tạo ra). Vì là các tiến trình độc lập, chúng không chia sẻ không gian địa chỉ và có thể chạy trên các nhân xử lý khác nhau của CPU.
- Controller không phải là một đoạn script nằm trong world .wbt. Nó là một chương trình riêng, sẽ được biên dịch, sẽ được biên dịch (C/C++), chạy như một process độc lập. Robot chỉ gọi chương trình này để lấy lệnh điều khiển

Bây giờ, chúng ta sẽ thực hành tạo một bộ điều khiển (controller) mới:

Bước 1: Tạo tệp điều khiển

- Vào menu **File / New / New Robot Controller...**
- Chọn ngôn ngữ **C** (hoặc ngôn ngữ bạn thích). Sau đó nháy Next
- Đặt tên cho bộ điều khiển là **epuck_go_forward** (Nếu dùng C++ hoặc Java, hãy đặt tên theo chuẩn CamelCase là **EPuckGoForward**).
- Thao tác này sẽ tạo một thư mục cùng tên trong đường dẫn **my_first_simulation/controllers**.



```

1 /*
2  * File:          epuck_go_forward.c
3  * Date:
4  * Description:
5  * Author:
6  * Modifications:
7  */
8
9 /*
10 * You may need to add include files like <webots/distance_
11 * <webots/motor.h>, etc.
12 */
13 #include <webots/robot.h>
14
15 /*
16 * You may want to add macros here.
17 */
18 #define TIME_STEP 64
19
20 /*
21 * This is the main program.
22 * The arguments of the main function can be specified by t
23 * "controllerArgs" field of the Robot node
24 */
25 int main(int argc, char **argv) {
26     /* necessary to initialize webots stuff */
27     wb_robot_init();
28
29     /*
30     * You should declare here WbDeviceTag variables for stor
31     * robot devices like this:
32     * WbDeviceTag my_sensor = wb_robot_get_device("my_sensa
33     * WbDeviceTag my_actuator = wb_robot_get_device("my_act
34     */
35
36     /* main loop
37     * Perform simulation steps of TIME_STEP milliseconds
38     * and leave the loop when the simulation is over
39     */
40     while (wb_robot_step(TIME_STEP) != -1) {
41         /*

```

Hình minh họa

Bước 2: Mở trình soạn thảo

- Chọn tùy chọn mở tệp nguồn (source file) bằng trình soạn thảo văn bản (**text editor**).
- Tệp nguồn mới sẽ hiển thị trong cửa sổ soạn thảo của Webots.
- Mã nguồn này có thể biên dịch ngay (nếu là C, C++ hoặc Java) mà không cần chỉnh sửa gì, tuy nhiên hiện tại nó chưa có tác dụng điều khiển robot thực tế.

Bước 3: Liên kết Controller với Robot

Bây giờ, chúng ta sẽ liên kết bộ điều khiển `epuck_go_forward` vừa tạo vào Node **E-puck** trong Scene Tree:

1. Trong cửa sổ **Scene Tree**, tìm đến Node **E-puck** (có thể cần cuộn xuống dưới cùng).
2. Mở rộng nút **E-puck** bằng cách nhấn vào dấu mũi tên bên cạnh.
3. Tìm trường có tên là `controller`.
4. Nhấp vào giá trị hiện tại (thường là `"void"` hoặc một tên mặc định khác) và nhấn nút **Select...**
5. Chọn `epuck_go_forward` từ danh sách hiện ra và nhấn **OK**.



Hình minh họa

C. Cho Robot hoạt động theo bộ điều khiển

Bây giờ, hãy chỉnh sửa chương trình bằng cách thêm thư viện điều khiển động cơ và thiết lập vị trí cho động cơ theo đoạn mã sau:

```

#include <webots/robot.h>

// Added a new include file
#include <webots/motor.h>

#define TIME_STEP 64

int main(int argc, char **argv) {
    wb_robot_init();

    // get the motor devices
    WbDeviceTag left_motor = wb_robot_get_device("left wheel motor");
    WbDeviceTag right_motor = wb_robot_get_device("right wheel motor");
    // set the target position of the motors
    wb_motor_set_position(left_motor, 10.0);
    wb_motor_set_position(right_motor, 10.0);

    while (wb_robot_step(TIME_STEP) != -1);

    wb_robot_cleanup();

    return 0;
}

```

Đoạn mã cho robot tiến thẳng

Tips:

Bạn có thể đổi controller

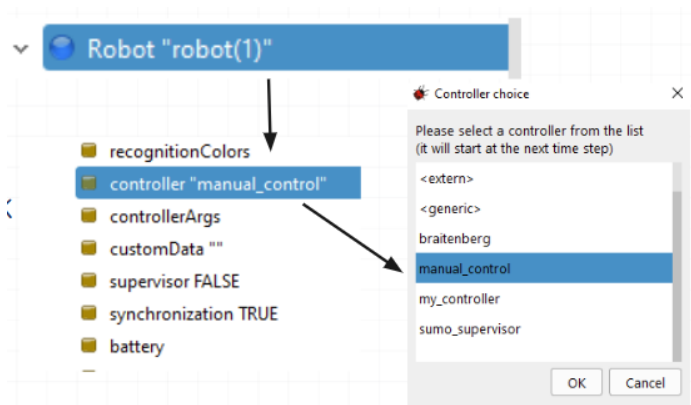
Dùng khi:

Chuyển chế độ tự động sang điều khiển tay (và ngược lại)

Tạo lập chương trình mới

Hướng dẫn đổi controller (slide 12):

 Webot Installation & Simple Usage Guide (2025).pptx



Các bước thực hiện tiếp theo:

1. **Lưu mã nguồn:** Vào menu **File / Save Text File**.
2. **Biên dịch:** Vào menu **Build / Build** (hoặc nhấn biểu tượng bánh răng). Hãy sửa lỗi cú pháp nếu có.

3. **Khởi động lại:** Khi Webots hỏi bạn có muốn reset hoặc tải lại thế giới không, hãy chọn **Reset** và chạy mô phỏng.

Kết quả quan sát được:

Nếu mọi thứ hoạt động bình thường, robot của bạn sẽ di chuyển về phía trước. Nó sẽ chạy với tốc độ tối đa trong một thời gian ngắn và dừng lại khi bánh xe đã quay đủ **10 radian**.

Lưu ý về cấu trúc thư mục:

- Trong thư mục `controllers` của dự án, một thư mục con tên là `epuck_go_forward` đã được tạo ra.
- Bên trong đó chứa tệp nhị phân (binary) được tạo sau khi biên dịch (trên Windows, tệp này có đuôi `.exe`).
- **Quy tắc quan trọng:** Tên thư mục của bộ điều khiển phải trùng khớp với tên của tệp nhị phân bên trong.

IV. Mở Rộng Và Kết Luận

A. Mở rộng sang điều khiển tốc độ

Các bánh xe của robot thường được điều khiển bằng vận tốc, chứ không phải bằng vị trí như chúng ta đã làm trong ví dụ trước đó. Để điều khiển tốc độ của các động cơ bánh xe, bạn cần thiết lập vị trí mục tiêu là vô cực và đặt tốc độ mong muốn thông qua đoạn mã sau (cố gắng gõ lại hoặc tốt hơn là dùng kiến thức và tự code cho mình nhé):


```

#include <webots/robot.h>

// Added a new include file
#include <webots/motor.h>

#define TIME_STEP 64

#define MAX_SPEED 6.28

int main(int argc, char **argv) {
    wb_robot_init();

    // get a handler to the motors and set target position to infinity (speed control)
    WbDeviceTag left_motor = wb_robot_get_device("left wheel motor");
    WbDeviceTag right_motor = wb_robot_get_device("right wheel motor");
    wb_motor_set_position(left_motor, INFINITY);
    wb_motor_set_position(right_motor, INFINITY);

    // set up the motor speeds at 10% of the MAX_SPEED.
    wb_motor_set_velocity(left_motor, 0.1 * MAX_SPEED);
    wb_motor_set_velocity(right_motor, 0.1 * MAX_SPEED);

    while (wb_robot_step(TIME_STEP) != -1) {
    }

    wb_robot_cleanup();

    return 0;
}

```

Đoạn mã điều chỉnh tốc độ

Với đoạn mã trên, giờ đây robot sẽ di chuyển (các bánh xe sẽ quay với tốc độ 0,2 radian trên giây) và không bao giờ dừng lại. Nếu không có hiện tượng gì xảy ra, đừng quên biên dịch mã nguồn của bạn bằng cách chọn mục Build / Build trong menu hoặc nhấp vào biểu tượng bánh răng phía trên vùng soạn thảo mã. Các lỗi biên dịch sẽ được hiển thị bằng màu đỏ ở cửa sổ console. Nếu có lỗi, hãy sửa chúng và thử biên dịch lại. Sau đó, hãy tải lại (reload) thế giới mô phỏng.

Kết Luận

Bên cạnh đó, mình và bạn cùng tổng hợp nhẹ lại các kiến thức đã học trong buổi này nhé:

1. Một thế giới (world) được cấu tạo từ các nút (nodes) sắp xếp theo cấu trúc cây.
2. Thế giới được lưu trong tệp .wbt nằm trong một dự án Webots.
3. Dự án cũng chứa các chương trình điều khiển robot (controllers) nhằm xác định hành vi của chúng.
4. Bộ điều khiển có thể được viết bằng ngôn ngữ C hoặc các ngôn ngữ khác.
5. Các bộ điều khiển bằng C, C++ và Java phải được biên dịch một cách rõ ràng trước khi có thể thực thi.
6. Bộ điều khiển được liên kết với robot thông qua trường 'controller' của nút Robot."

Team Simulation - Robotics

Liên Hệ thành viên hỗ trợ:

hoanghuy.pham@steamforvietnam.org

0363160442 (Zalo - Hoàng Huy)

Cảm ơn các bạn đã trải nghiệm tài liệu của mình. Hi vọng các bạn có các giờ học thật bổ ích !!